

AI Enhanced Project Management Platform

Project Details

Role: Software Developer

Project Type: fullstack

Difficulty: senior

Duration: 365 days

Focus: api_development, authentication, database, fullstack, problem_solving, testing, security, microservices, real_time, cloud, performance, system_design, ai_integration, frontend

Student Name: Rahul Sharma

Description: A full stack platform for teams to plan, track, and analyze projects, with AI driven task recommendations and real time collaboration.

Technologies

- JavaScript
- TypeScript
- React
- Next.js
- Node.js
- Express.js
- Laravel
- Tailwind CSS
- MongoDB
- MySQL
- FastAPI
- Python
- NumPy
- AWS
- Groq API
- JWT

Requirements

- Use existing skills for core features.
- Introduce Python microservice for AI.
- Deploy on AWS with CI/CD.
- Implement role based auth and real time updates.

Objectives

- Design and implement a microservice based architecture.
- Provide a responsive Next.js UI with Tailwind.
- Integrate AI task recommendation via FastAPI and Groq.
- Ensure security, testing, and production grade deployment.

Functional Requirements

- Users register, login, and receive JWTs.
- Role based access: Admin, Manager, Contributor.
- Create, edit, delete projects and tasks.
- Real time task updates via WebSocket.
- AI endpoint suggests next tasks based on project data.
- Export project reports as PDF.

Non Functional Requirements

- Password hashing with bcrypt.
- Rate limiting on public APIs.
- Response time < 200 /ms for core endpoints.
- 99.5 /% uptime on AWS.

- Comprehensive unit, integration, and E2E tests.

Modules

Auth Service

Description: Handles registration, login, JWT issuance, and RBAC.

Features

- JWT auth
- Password reset
- Role checks

Project API

Description: CRUD for projects and task hierarchy.

Features

- Project CRUD
- Task CRUD
- Bulk operations

Realtime Engine

Description: WebSocket server for live task updates.

Features

- Socket rooms
- Change broadcast
- Presence tracking

AI Recommendation Service

Description: Python FastAPI microservice using Groq LLM to suggest tasks.

Features

- Feature extraction
- LLM prompt
- Result caching

Frontend UI

Description: Next.js app with Tailwind, supporting dashboards and editors.

Features

- Responsive layout
- State management
- PDF export

Admin Portal

Description: Laravel backend for user and role management.

Features

- User admin
- Audit logs
- Metrics view

Data Layer

Description: MongoDB for project data, MySQL for relational reports.

Features

- Schema design
- Indexes
- Backup scripts

CI/CD Pipeline

Description: GitHub Actions deploying to AWS ECS/Fargate.

Features

- Linting
- Test run
- Blue green deploy

Implementation Plan

Foundation

Objective: Set up repo, CI, and basic infra.

Milestone: Repository with CI pipeline and AWS account ready.

Tasks

- Initialize monorepo with Yarn workspaces
- Configure GitHub Actions for lint and test
- Provision VPC, RDS (MySQL), and DocumentDB (MongoDB) on AWS
- Create Dockerfiles for Node and Python services

Deliverables

- Repo structure
- CI config
- AWS infra scripts

Core Backend

Objective: Implement Auth and Project APIs.

Milestone: Secure REST endpoints with JWT.

Tasks

- Build Express auth routes
- Add role based middleware
- Create Project CRUD routes
- Write unit tests with Jest

Deliverables

- Auth service
- Project API
- Test suite

Frontend MVP

Objective: Deliver basic UI for project management.

Milestone: Users can manage projects and tasks.

Tasks

- Scaffold Next.js with TypeScript
- Integrate Tailwind CSS
- Implement login flow and protected routes
- Build project/task list components

Deliverables

- Login page
- Dashboard
- Task editor

AI Integration

Objective: Add task recommendation microservice.

Milestone: AI endpoint returns suggestions.

Tasks

- Create FastAPI service with Groq API call
- Define data contract between services
- Add caching layer (Redis) for suggestions
- Write integration tests

Deliverables

- AI service
- API docs
- Integration tests

Security & Performance

Objective: Harden system and optimize latency.

Milestone: Pass security audit and meet <200 /ms SLA.

Tasks

- Implement rate limiting (express-rate-limit)
- Add input validation (Joi)
- Profile endpoints, add indexes
- Configure CloudWatch alarms

Deliverables

- Security report
- Performance benchmarks

Production Deployment

Objective: Release stable version with monitoring.

Milestone: Live system on AWS with zero downtime deploy.

Tasks

- Set up ECS services and load balancer
- Configure Route53 and SSL
- Enable centralized logging (ELK)
- Run full regression suite

Deliverables

- Deployed app
- Monitoring dashboards
- Rollback plan

Evaluation Criteria

Architecture

Description: Coherent microservice design

Weight: 20

Core Functionality

Description: All functional requirements met

Weight: 20

Security

Description: Proper auth, validation, hardening

Weight: 15

Testing

Description: Coverage and test quality

Weight: 15

Performance

Description: Meets response time SLA

Weight: 15

Documentation & Deployability

Description: Clear docs, CI/CD works

Weight: 15

Deliverables

- Source code repository
- API specification (OpenAPI)
- Database schema diagram
- CI/CD pipeline configuration
- Test suite with coverage report
- Deployment guide and monitoring dashboards

Api Endpoints

POST

Path: /api/auth/register

Purpose: Create new user

POST

Path: /api/auth/login

Purpose: Issue JWT

GET

Path: /api/projects

Purpose: List user projects

POST

Path: /api/projects

Purpose: Create project

GET

Path: /api/tasks/:id

Purpose: Get task details

POST

Path: /api/ai/recommend

Purpose: Get AI task suggestions

Student Fit: Rahul's strong JavaScript/TypeScript, React, Node, Express, and JWT experience aligns with the core stack. Adding a Python FastAPI AI microservice, AWS deployment, and advanced security/testing expands his skill set while staying within his senior level capability.

Student Id: 6ab2576039d4e38828d96a4e

Status: active